

KAREL

Huiswerk 3: Oefentoets

Naam: _____

Instructies. Dit is een oefententamen van dezelfde vorm als de tussentoets in sessie 4. Je hebt er 90 minuten voor.

Maak het oefententamen zoals je een echt tentamen maakt: één lange zit, geen computer, geen hulp, en stoppen als de tijd om is. Pas daarna ga je je werk nakijken en met anderen vergelijken.

De toolbox op de volgende pagina krijg je ook bij de tussentoets.

Vraag	Wat er van je gevraagd wordt	Punten
1. Traceer de robot	Lees een programma; zeg precies wat er gebeurt	0,6
2. Vereenvoudig	Vereenvoudig twee instructies, en benoem de regels	0,6
3. Herontwerp	Herschrijf een eenvoudig programma naar een nette versie	0,9
4. Los het op	Schrijf een programma voor een variabele wereld	0,9
Totaal		3

Punten. Een beperkt opgedeeld programma (de refinement) levert minder punten op dan een programma met een korte main en korte instructies. Er wordt dus om nette en overzichtelijke code gevraagd! Een klein foutje in de correctheid is dan weer niet zo'n probleem. Puntkomma's moeten wél op de juiste plaatsen gebruikt worden.

Toolbox

Instructies die Karel al kent. Gebruik ze allemaal vrij.

Basis	move, turnleft, pickbeeper, putbeeper, turnoff
Draaien	turnright, turnaround
Kijkrichting	face-north, face-east, face-south, face-west

De tests die Karel kan uitvoeren.

Is de weg versperd?	front-is-clear / front-is-blocked left-is-clear / left-is-blocked right-is-clear / right-is-blocked
Ligt hier een beeper?	next-to-a-beeper / not-next-to-a-beeper
Zit er iets in de bag?	any-beepers-in-beeper-bag / no-beepers-in-beeper-bag
Welke kant kijk ik op?	facing-north / not-facing-north facing-east / not-facing-east facing-south / not-facing-south facing-west / not-facing-west

De vier ontwerpgereedschappen

IF <test> THEN <instruction>	IF <test> THEN <instruction> ELSE <instruction>
ITERATE n TIMES <instruction>	WHILE <test> DO <instruction>

Overall waar één instructie is toegestaan, mag ook een BEGIN ... END-groep met meer instructies staan. Een ELSE hoort bij de dichtstbijzijnde IF erboven die er nog geen heeft.

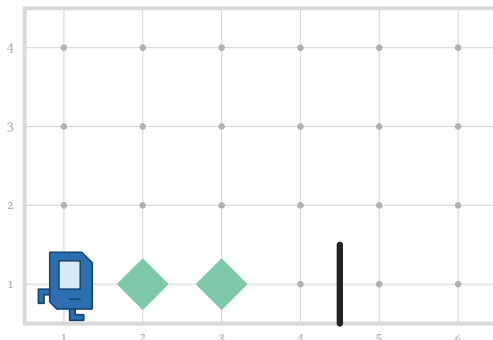
De vier transformaties

- **Test reversal.** Draai de test om, wissel de takken. Is een tak leeg, laat dan de ELSE vallen.
- **Top factoring.** Beide takken beginnen met dezelfde instructie. Haal die uit de takken en zet hem boven de IF.
- **Bottom factoring.** Beide takken eindigen met dezelfde instructie. Haal die uit de takken en zet hem onder de IF/ELSE.
- **Redundant-test factoring.** Als je door de code te traceren al weet hoe de test uitpakt: verwijder de test en herschrijf de rest.

1 Traceer de robot

0,6 punten

Karel start in de wereld hieronder, op coördinaat (1,1) en kijkend naar het oosten. Een muur blokkeert de oostkant van de corner (4,1). Twee beepers liggen op de grond. Het algemene doel: veeg alle beepers van de zuidelijkste straat weg, van linksonder tot aan de muur, en navigeer dan recht omhoog naar het noorden. Het programma moet enigszins flexibel zijn: het moet correct werken waar de muur ook ligt, hoeveel beepers er ook zijn, en hoe hoog de wereld ook is.



```
BEGINNING-OF-PROGRAM
DEFINE-NEW-INSTRUCTION scoop AS
  IF next-to-a-beeper THEN
    pickbeeper;
BEGINNING-OF-EXECUTION
  WHILE front-is-clear DO
  BEGIN
    scoop;
    move
  END;
  scoop;
  turnleft;
  WHILE front-is-clear DO
    move;
  END-OF-EXECUTION
END-OF-PROGRAM
```

- (a, 0,2 pt) Traceer het programma. Markeer Karels eindpositie en kijkrichting rechtstreeks op de kaart hierboven, en geef aan hoeveel beepers Karel bij zich draagt op het moment dat hij uitschakelt.

- (b, 0,1 pt) Is dit een normale shutoff of een error-shutoff? Zeg hoe je dat weet. Als het een error-shutoff is, wat moet er dan gerepareerd worden?

- (c, 0,2 pt) Waarom is de tweede scoop (die na de loop, op een eigen regel) nodig? Beschrijf precies de beeper die hij moet oppakken.

- (d, 0,1 pt) Stel dat de muur ten oosten van (4,1) wordt weggehaald en al het andere gelijk blijft. Haalt het programma dan nog steeds zijn doelen? Zo niet, wat gaat er mis? Zo ja, welk deel van het programma zorgt voor de flexibiliteit om ook in dat geval te werken?

2 Vereenvoudig

0,6 punten

Laat je werk zien. Benoem bij elke stap welke transformatie je gebruikt en schrijf op hoe de instructie er dan uitziet. Punten krijg je voor correct benoemde én correct toegepaste transformaties. Combineer je meerdere transformaties in één stap (zonder je werk te laten zien), dan krijg je daar geen punten voor.

→ **(a, 0,3 pt)** Vereenvoudig de instructie approach zo ver als het gaat. Laat alle stappen zien, met de naam van de vereenvoudiging of een korte uitleg.

DEFINE-NEW-INSTRUCTION approach AS

BEGIN

IF front-is-clear THEN

BEGIN

```
move;
```

pickbeeper

END

ELSE

BEGIN

```
turnright;
```

```
move;
```

pickbeeper

END

END

This image shows a single sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

↪ **(b, 0,3 pt)** Vereenvoudig swing-north.

```

DEFINE-NEW-INSTRUCTION swing-north AS
BEGIN
    IF not-facing-south THEN
    BEGIN
        putbeeper;
        move
    END
    ELSE
    BEGIN
        putbeeper;
        turnleft;
        IF facing-east
            THEN
                turnleft
            ELSE
                turnleft;
        move
    END
END

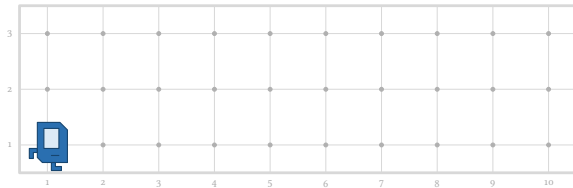
```

This image shows a single sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

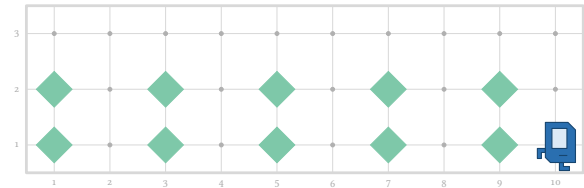
3 Herontwerp

0,9 punten

Karel bouwt een hekje. Hij start op (1,1), kijkend naar het oosten, en moet op elke oneven avenue een beeper zetten op straat 1 én straat 2 (een paaltje), zoals in het voorbeeld hieronder. Hij eindigt helemaal rechts, hier op (10,1), kijkend naar het oosten.



Voor: kale grond.



Na: vijf paaltjes neergezet.

Een student levert het onderstaande programma in. Het is **correct**, maar tegelijk niet echt **goed**: de code is heel lang en heel lastig te traceren. Jouw taak: verbeter de netheid en overzichtelijkheid van de code, en zorg dat deze tegelijk correct blijft werken.

BEGINNING-OF-PROGRAM	turnleft;	turnleft;
BEGINNING-OF-EXECUTION	move;	move;
putbeeper;	move;	turnleft;
turnleft;	putbeeper;	move;
move;	turnleft;	move;
putbeeper;	move;	putbeeper;
turnleft;	putbeeper;	turnleft;
turnleft;	turnleft;	move;
move;	turnleft;	putbeeper;
turnleft;	move;	turnleft;
move;	turnleft;	turnleft;
move;	move;	move;
putbeeper;	move;	turnleft;
turnleft;	putbeeper;	move;
move;	turnleft;	turnoff
putbeeper;	move;	END-OF-EXECUTION
turnleft;	putbeeper;	END-OF-PROGRAM
turnleft;	turnleft;	
move;		

→ (a, 0,1 pt) Op basis van je ervaring zou je twee duidelijke hoofdtaken in het programma moeten kunnen aanwijzen. Ze komen beide meerdere keren voor. Beschrijf deze twee taken, en markeer voor elk een voorbeeld in de code hierboven (schrijf de naam er ook bij voor de duidelijkheid).

⤵ **(b, 0,3 pt)** Hou die twee taken uit de vorige vraag in gedachten en ontwerp een refinement. Gebruik niet alléén die twee taken! Splits ze verder op waar dat kan. Je hoofdprogramma moet uiteindelijk heel kort zijn, maar elke losse zelfgedefinieerde instructie het liefst ook.

Instructienaam	Waar het voor verantwoordelijk is

→ **(c, 0,5 pt)** Schrijf het **main**-programma volledig uit, met de instructies die je in je refinement hebt gedefinieerd. Het geheel moet ruimschoots in de ruimte hieronder passen; zo niet, verfijn dan je ontwerp.

[illegible]

Als we het programma in een bredere wereld gebruiken, moet die nog steeds vol paaltjes komen te staan, niet alleen de vijf van eerder. Houdt je programma daar al goed rekening mee? Zo niet, leg dan duidelijk uit hoe je dit zou moeten oplossen. Is het al goed, leg dan duidelijk uit waarom.

→ **(c, 0,5 pt)** Schrijf het hele programma: BEGINNING-OF-PROGRAM, je definities, het main-executieblok, END-OF-PROGRAM. Elke instructie die niet in de toolbox staat (zie eerder in dit boekje) moet je zelf schrijven.

Stel, Karel wordt neergezet in een gang *zonder ook maar één beeper*, of in een gang waar juist de allereerste corner (1,1) een stapel heeft. Werkt je programma in allebei zonder ook maar één wijziging? Leg goed uit. Als je programma (nog) niet werkt in die situaties, leg dan concreet uit hoe je programma aangepast moet worden (je hoeft niet het complete programma opnieuw te schrijven).

[illegible]

Check nu je puntkomma's in al je uitwerkingen.